



Stellenbosch
UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

Probabilistic Modelling and Reasoning - 812

Bayesian Neural Networks

DJ Swanevelder

March 24, 2025

Department of Applied Mathematics
Stellenbosch University

Abstract

Bayesian Neural Networks (BNNs) apply Bayesian uncertainty modeling to the training of neural network weights, offering both advantages and challenges. This report first establishes the necessary theoretical background for understanding BNNs, building on concepts from the PMR-812 course. The discussion is then extended beyond the course material to provide a deeper insight into the field. A practical implementation of BNNs is presented, demonstrating their application to a relevant problem. Finally, the report concludes with an analysis of the limitations, benefits, and future opportunities of BNNs.

Copyright © 2025 Stellenbosch University.
All rights reserved.

1 Introduction

1.1 Overview of Bayesian Neural Networks

Bayesian Neural Networks (BNNs) are a probabilistic extension of traditional Neural Networks (NNs), incorporating Bayesian inference to estimate uncertainty in predictions. Unlike standard NNs, which learn a single set of weights, BNNs learn a distribution over weights, capturing the uncertainty in model parameters. This allows BNNs to generate a predictive distribution for each input, rather than a single deterministic output.

One of the key advantages of BNNs is their ability to provide both a prediction and a measure of uncertainty. While some applications prioritize raw accuracy, others require models to be highly confident before making a decision—or to abstain if uncertainty is too high.

This property is particularly valuable in high-stakes applications where incorrect predictions can have severe consequences. For example, in medical diagnosis, a BNN detecting disease can indicate when its confidence is low, prompting further review by a doctor rather than risking a misdiagnosis. Similarly, in autonomous driving, if a BNN is uncertain about detecting an obstacle, the vehicle can default to a safer action, such as slowing down, rather than making an uninformed decision. In contrast, a standard NN would always output a prediction, even when uncertain, increasing the risk of critical errors.

1.2 Context within the Course Framework

BNNs treat the weights of a standard NN as random variables, for which a posterior distribution is learned. This posterior is defined using Bayes' rule:

$$P(w|\mathcal{D}) = \frac{p(\mathcal{D}|w)p(w)}{p(\mathcal{D})}$$

However, solving for the posterior exactly is intractable, as computing the evidence $p(\mathcal{D})$ requires integrating over all possible values of w . While numerical approximations for solving this integral exist, the computational cost scales exponentially with the dimensionality and depth of the network.

To determine the posterior, approximate probabilistic inference methods are required. Monte Carlo methods, though widely used, often require an impractically large number of samples to be effective for BNNs. Similarly, Markov Chain Monte Carlo (MCMC) methods can be inefficient for deep networks. Variational inference (VI), however, provides a more practical and computationally efficient approach for BNNs.

Variational inference approximates the posterior with a simpler, parameterized distribution, enabling more efficient computation. This approach requires defining both the prior and posterior distributions, which inherently regularizes the network, preventing overfitting while maintaining uncertainty estimation.

2 Theory and Explanation

2.1 Variational Infererence

To determine the posterior $p(w|\mathcal{D})$, an approximate surrogate distribution $q(w)$ is introduced. The goal of Variational Inference (VI) is to ensure that this approximate posterior closely matches the true posterior. This is achieved by minimizing the Kullback-Leibler (KL) divergence, leading to the optimal choice of $q^*(w)$:

$$q^*(w) = \arg \min_q KL_{q(w)}(q(w)||p(w|\mathcal{D})) = \langle \log q(w) \rangle_{q(w)} - \langle \log p(\mathcal{D}, w) \rangle_{q(w)}$$

Minimizing the KL divergence is equivalent to maximizing its negative, known as the Evidence Lower Bound (ELBO). Using the factorization $p(\mathcal{D}, w) = p(\mathcal{D}|w) \cdot p(w)$, we derive:

$$\mathcal{L}(q) = \langle \log p(\mathcal{D}, w) \rangle_{q(w)} - \langle \log q(w) \rangle_{q(w)} \quad (1)$$

$$= \langle \log p(\mathcal{D}|w) \rangle_{q(w)} + \langle \log p(w) - \log q(w) \rangle_{q(w)} \quad (2)$$

$$= \langle \log p(\mathcal{D}|w) \rangle_{q(w)} - KL_{q(w)}(q(w)||p(w)) \quad (3)$$

Maximizing the ELBO corresponds to maximizing the log-likelihood while incorporating a penalty term that constrains deviations from the prior. This inherently regularizes the posterior distribution, ensuring it remains close to the prior, and often results in an effect similar to conventional regularization techniques employed in standard neural networks.

The specific form of the ELBO depends on the choice of the variational distribution q , such as a Gaussian distribution. To optimize the parameters of q and maximize the ELBO, Stochastic Variational Inference (SVI) can be utilized. SVI updates the variational parameters iteratively using mini-batches of data, significantly improving computational efficiency compared to traditional full-batch variational inference.

2.2 Modelling Uncertainty

Once the approximate posterior distribution over the weights $p(w|\mathcal{D})$ is found for all weights in the network, we can determine a predictive distribution for any given input x . Given that $y(x)$ represents the network's output, the predictive distribution is defined as:

$$p(y(x)|\mathcal{D}) = \int_w p(y(x)|w)p(w|\mathcal{D})dw = \langle p(y(x)|\mathcal{D}) \rangle_{p(w|\mathcal{D})}$$

This predictive distribution can be interpreted as an infinite ensemble of neural networks [1], where each model's contribution is weighted according to the probability of its weights, $p(w|\mathcal{D})$. In other words, instead of having a single deterministic model making predictions, the BNN accounts for uncertainty in its weights by averaging over infinitely many possible networks.

Since computing this integral exactly is infeasible, we approximate it using Monte Carlo (MC) sampling. This is done by drawing multiple samples of the network’s weights from the posterior $p(w|\mathcal{D})$. For each sampled set of weights, the network performs a forward pass on the input x , producing a set of possible outputs. Mathematically, this is written as:

$$p(y(x)|\mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S p(y(x)|w_s), \quad w_s \sim p(w|\mathcal{D})$$

where S is the number of Monte Carlo samples drawn from the posterior [2]. Each sampled weight configuration can be thought of as drawing a specific neural network from the ensemble, with its selection probability determined by the posterior distribution. The final predictive distribution is then formed by aggregating predictions from all sampled models.

Once the predictive distribution is obtained for a given input, the model output is typically chosen as:

- The **mean** of the distribution in regression tasks.
- The **mode** (i.e., the most frequent prediction) in classification tasks.

However, unlike traditional neural networks, Bayesian Neural Networks provide more than just point estimates, they yield an entire probability distribution over predictions. This enables an assessment of the model’s confidence and uncertainty.

Figure 1 illustrates predictive distributions for a regression task. Both distributions have the same mean and thus produce the same final model output. However, the dashed distribution has a significantly larger variance, meaning that its predictions are more spread out. This implies that the model is less certain about its prediction, as there is a wider range of likely values. In contrast, the solid distribution has a narrower spread, indicating higher confidence in the prediction.

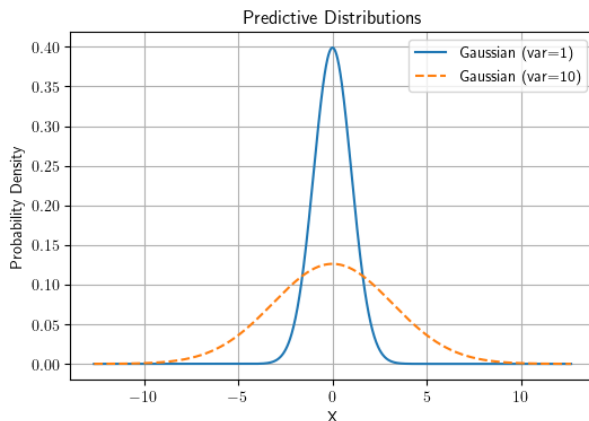


Figure 1: Examples of predictive distributions for regression

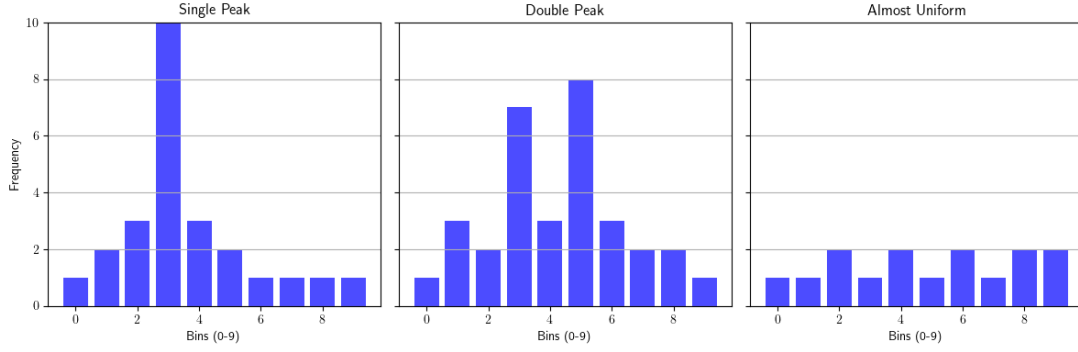


Figure 2: Examples of predictive distributions for classification

This demonstrates why the variance of the predictive distribution is often interpreted as a measure of the model’s uncertainty, higher variance implies greater uncertainty, whereas lower variance suggests a more confident prediction.

Figure 2 illustrates three possible predictive distributions for a classification task. In classification, the frequency of the most commonly predicted class (the mode frequency) is often used as a measure of confidence:

- The first distribution has a strong peak at class 3, meaning most sampled networks predict class 3. This suggests high confidence in the classification.
- The second distribution has two peaks, indicating uncertainty between two possible classes. This suggests the model is unsure which class is correct.
- The third distribution has no distinct peak, meaning predictions are spread across many classes. This indicates significant uncertainty, as no single class is strongly preferred.

Thus, in classification, mode frequency serves as a useful proxy for confidence: a sharp peak suggests high certainty, while a flatter distribution implies greater uncertainty.

2.3 Modeling Confidence

The variance and mode frequency are often a simple and fast way to estimate the model’s confidence in its prediction. However, these metrics do not utilize all the information available in the predictive distribution. To address this, an additional model is proposed that takes the predictive distribution as input and learns its relationship to the model’s accuracy on the training data. The assumption is that this learned relationship generalizes to unseen data.

The goal of this model is to provide a confidence measure that represents the conditional probability that the BNN output y is correctly classified, given the predictive distribution. Let C be a random variable that is 1 if the model correctly classifies the input and 0 otherwise. Then, the confidence function can be expressed as:

$$\text{confidence}(x) = p(C = 1 \mid p(y(x) \mid \mathcal{D}))$$

Focusing on classification, there are several intuitive parameters that can be used to estimate the certainty of the model’s prediction for a given input:

- F_{m1} : The mode frequency, which indicates whether there is a clear winner among the classifications.
- U : The number of unique classifications. A higher U suggests greater uncertainty, as the predictions are spread over many classes.
- F_{m2}, F_{m3} : The second and third mode frequencies, which help detect cases where there is a tie or multiple competing classifications.

Note that the frequencies are discrete variables, having a step size of $1/S$, where S is the number of samples.

The confidence model is then simplified to:

$$\text{confidence}(x) = p(C = 1 \mid F_{m1}, F_{m2}, F_{m3}, U)$$

A graphical representation is shown in Figure 3, where M_i represents the samples that collectively form the predictive distribution.

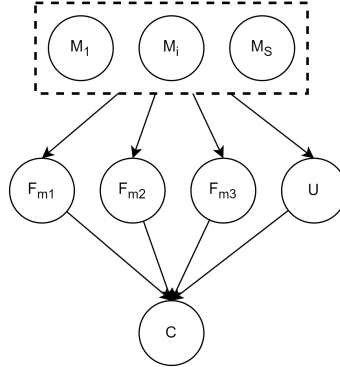


Figure 3: Graphical Representation of the Proposed Confidence Model

To estimate the conditional probability tables for all possible combinations of F_{m1}, F_{m2}, F_{m3} , and U , traditional maximum likelihood estimation (MLE) is used. These estimated probabilities are then leveraged to generate a confidence measure that captures more aspects of the predictive distribution than simply the mode frequency. The goal is to have a threshold confidence, above which almost no misclassifications occur.

3 Case Study: MNIST Classification

The objective of this case study is to evaluate a basic Bayesian Neural Network (BNN) and its ability to reliably determine confidence in its predictions.

3.1 Model Architecture

To assess the uncertainty modeling capabilities of BNNs, a classifier is trained on the well-known MNIST dataset [3]. The model architecture is intentionally kept as simple as possible due to the complexity involved in setting up a BNN. Table 1 presents the model’s architecture. Each MNIST image (28×28) is flattened before being fed into the input layer, resulting in an input dimension of 784.

The dataset is split into three subsets: **training (64%)**, **validation (16%)**, and **testing (20%)**. The model’s accuracy and loss are computed on the validation set after each epoch, with early stopping applied—if the validation loss does not improve for three consecutive epochs, training is halted.

The approximate posterior distribution $q(w)$ is modeled as a diagonal-covariance Gaussian:

$$q(w) = \mathcal{N}(\mu, \sigma^2 \cdot \mathbf{I})$$

Stochastic Variational Inference (SVI) is used for training, with the Evidence Lower Bound (ELBO) serving as the loss function. Optimization is performed using the Adam optimizer. The implementation is done in **PyTorch**, inspired by [4], and the full code can be found in [5].

Layer	Neurons	Activation	Prior
Input	784	-	-
Hidden Layer	784	ReLU	$\mathcal{N}(\mu, \sigma^2 \mathbf{I})$
Output	10	SoftMax	-

Table 1: MNIST Classifier Model Architecture

3.2 Model Training

The model was trained, and the validation and training loss per epoch are shown in Figure 4. After training, 10 Monte Carlo (MC) samples are generated for each training data instance, approximating the predictive distribution. The mode of this distribution is taken as the predicted class and compared to the true class to determine the value of C .

For each training instance, the first, second, and third mode frequencies (F_{m1}, F_{m2}, F_{m3}) as well as the number of unique predicted classes (U) are computed and stored in a table, as shown in Table 2.

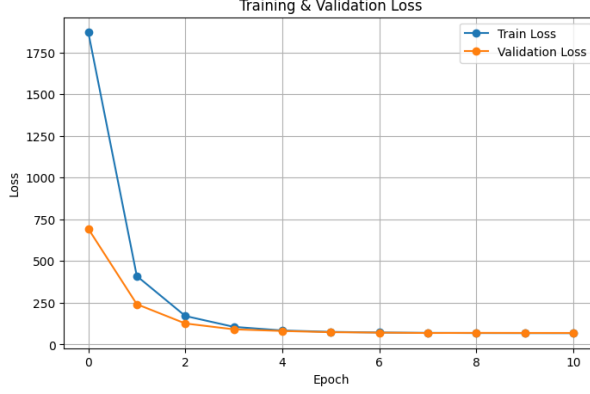


Figure 4: Graph of the Validation and Training Loss

Image Index	U	F_{m1}	F_{m2}	F_{m3}	C
0	1	1.0	0.0	0.0	1
1	3	0.8	0.1	0.1	1
2	2	0.9	0.1	0.0	1
3	1	1.0	0.0	0.0	1

Table 2: Confidence Model Input Data

The conditional probability table, or $\text{confidence}(x)$, is computed as described in Section 2.3. The resulting values are structured as shown in Table 3.

3.3 Results

First, the results from the training dataset were considered. Figure 5 presents a bar graph where the y-axis represents the number of samples, and the x-axis corresponds to the parameters of the confidence model. In the graph, red bars indicate misclassified images, while green bars represent correctly classified images.

Figure 5a illustrates the distribution of F_{m1} on the x-axis. From this figure, it is evident that the first mode provides a strong initial estimate for the model’s confidence.

Firstly, almost all samples with a first-mode frequency of 1 are correctly classified, while nearly all images with a mode frequency below 0.3 are misclassified. This aligns with the expected behavior of confidence estimation. However, there is room for improvement, as a significant number of correctly classified images have low confidence levels. This implies that if a threshold-based confidence measure were used—where predictions are abstained from below a certain confidence level—a large portion of the dataset would remain unclassified. This observation motivates the need for a more nuanced definition of confidence.

Figures 5b, 5c, and 5d further demonstrate that additional information can be leveraged in a confidence function. The model’s accuracy decreases as U increases and is inversely related to the second and third mode frequencies, F_{m2} and F_{m3} , respectively.

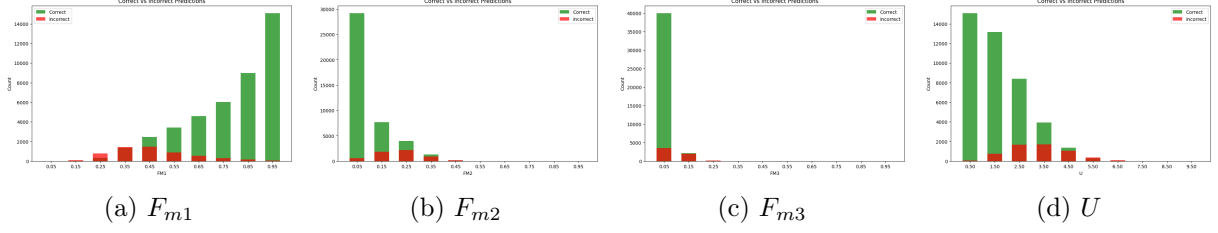


Figure 5: Feature distributions for confidence estimation.

The conditional probability tables were then learned, with key examples summarized in Table 3. The first entry aligns with expectations: if the model predicts only one unique class, it should have high confidence. Similarly, the second entry is intuitive—if two options are equally distributed, the confidence should be approximately 50%. The fourth entry further confirms the model’s behavior; when the number of unique classes is high, the model is almost certain that its prediction is incorrect. These probability tables provide a logical basis for confidence estimation. When applied to the test set, the resulting confidence distribution is shown in Figure 6.

U	F_{m1}	F_{m2}	F_{m3}	Confidence
1	1.0	0.0	0.0	99.6%
2	0.5	0.5	0.0	49.5%
2	0.4	0.3	0.3	41.8%
9	0.1	0.1	0.1	0.0%

Table 3: Subset of the conditional probability table.

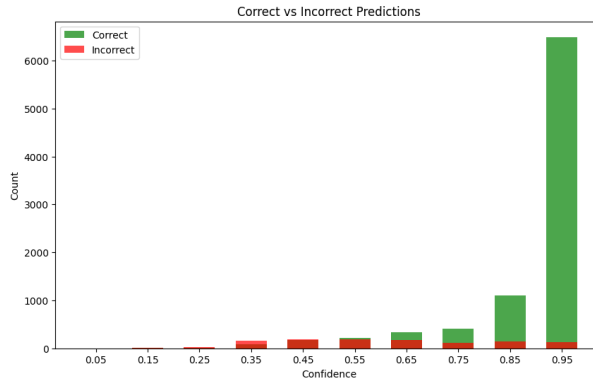


Figure 6: Image distribution of confidence values on the test set.

The key question that arises is: how useful is the ability to quantify a model’s confidence? Can accuracy be improved by setting a confidence threshold, below which the BNN abstains from classification? Figure 7 illustrates how accuracy increases as the confidence threshold is raised, alongside the proportion of classified data.

A model that achieves high accuracy by classifying only a small fraction of data is not practical. However, in certain applications, extreme accuracy is required. The baseline

accuracy of the simple BNN is 88.46% when using no confidence threshold ($\tau = 0$). Accuracy only begins to improve at $\tau = 0.3$, eventually reaching 99.76% at $\tau = 0.99$, though this threshold results in only 29.53% of the data being classified. This demonstrates the potential for achieving extremely high accuracy at the cost of reduced coverage. A more balanced threshold of $\tau = 0.68$ results in 97.26% accuracy while classifying 85% of the data, highlighting the trade-off between accuracy and the proportion of classified samples.

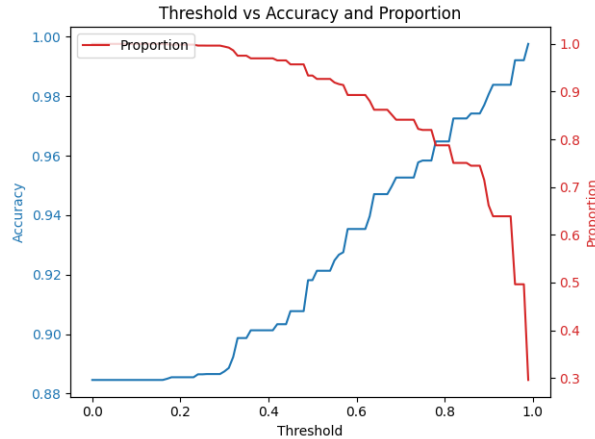


Figure 7: Model accuracy and classified proportion as a function of confidence threshold.

4 Conclusion

4.1 Limitations

One of the primary challenges of Bayesian Neural Networks (BNNs) is scalability. Due to the fact that BNN parameters are distributions rather than fixed values, they require significantly more parameters compared to traditional neural networks (NNs) with the same architecture. Additionally, a single forward pass in a BNN requires multiple forward passes, as the predictive distribution is approximated using multiple Monte Carlo samples drawn from the ensemble of models described by the BNN.

Although Stochastic Variational Inference (SVI) is an efficient training algorithm, the training time for BNNs is still considerably longer. However, this can be considered a one-time cost, with the benefit of obtaining uncertainty estimates for predictions throughout the model’s usage.

Another notable limitation is the programming complexity and support. Even with dedicated BNN packages, implementing and debugging a simple single-layer BNN required significantly more effort, debugging, and code compared to an equivalent standard NN.

4.2 Future Work

Further research can explore the regularization effects of different prior distributions in BNNs. For instance, a comparative study could analyze how well a BNN generalizes to

unseen data in contrast to a standard NN with the same architecture, both with and without traditional deep learning regularization techniques. Such an analysis could provide stronger justification for the use of BNNs.

Another promising avenue is the implementation of BNNs in state-of-the-art deep learning architectures, such as convolutional neural networks (CNNs). Given the observed improvement in accuracy using confidence-thresholded BNNs, extending this approach to Bayesian CNNs could yield highly accurate and reliable models.

An alternative approach worth exploring is leveraging BNN confidence scores to dynamically partition data. For instance, if a BNN exhibits low confidence for a subset of training images, these samples could be isolated and used to train a specialized BNN. This process could be iterated multiple times, with the number of iterations treated as a hyperparameter or determined by a predefined confidence threshold.

At inference time, an ensemble of BNNs could be deployed, with the model exhibiting the highest confidence making the final prediction. This framework could lead to a system where an initial generalist BNN is complemented by specialized BNNs trained on harder-to-classify examples, such as distinguishing between visually similar digits (e.g., 0 and 8). This approach bears resemblance to clustering methods, with the key distinction being that clusters emerge organically from the confidence scores rather than being learned through unsupervised learning methods.

Overall, BNNs offer significant advantages by not only providing predictions but also quantifying uncertainty. While modern machine learning research often prioritizes accuracy and computational efficiency, the growing integration of AI systems into critical areas of human life—where incorrect predictions can have serious consequences—necessitates models that offer both high accuracy and well-calibrated uncertainty estimates. Exploring and improving BNN methodologies may thus be crucial for the development of safer and more interpretable AI systems.

References

- [1] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, “Weight uncertainty in neural networks,” *International Conference on Machine Learning*, vol. 37, pp. 1613–1622, 2015. [Online]. Available: <http://proceedings.mlr.press/v37/blundell15.pdf>.
- [2] J. Yu, E. Creager, D. Duvenaud, and J. Bettencourt, *Bayesian neural networks*, 2015. [Online]. Available: https://www.cs.toronto.edu/~duvenaud/distill_bayes_net/public/.
- [3] L. Deng, “The mnist database of handwritten digit images for machine learning research [best of the web],” *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, Oct. 2012. DOI: 10.1109/msp.2012.2211477. [Online]. Available: <https://doi.org/10.1109/msp.2012.2211477>.
- [4] P. Chopra, *Paraschopra/bayesian-neural-network-mnist: Bayesian neural network using pyro and pytorch on mnist dataset*, 2025. [Online]. Available: <https://github.com/paraschopra/bayesian-neural-network-mnist>.
- [5] Swanevelder, *Pseudoboertjie/mnist_bayesian_nn: A mnist classifier using pytorch and a bayesian neural network*, 2025. [Online]. Available: https://github.com/Pseudoboertjie/MNIST_bayesian_NN.